

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

Лабораторная работа №4 по курсу
«Операционные системы»

Тема работы
“Динамические библиотеки”

Студент: Салихов Руслан Ринатович
Группа: М8О-203Б-23
Вариант: 32

Преподаватель: Миронов Евгений Сергеевич
Оценка: _____
Дата: _____
Подпись: _____

Москва, 2024

Постановка задачи

Задача: требуется создать динамические библиотеки, которые реализуют определенный функционал. Далее использовать данные библиотеки 2-мя способами:

- 1) Во время компиляции (на этапе «линковки»/linking)
- 2) Во время исполнения программы. Библиотеки загружаются в память с помощью интерфейса ОС для работы с динамическими библиотеками

В конечном итоге, в лабораторной работе необходимо получить следующие части:

- Динамические библиотеки, реализующие контракты, которые заданы вариантом;
- Тестовая программа (программа №1), которая использует одну из библиотек, используя знания, полученные на этапе компиляции;
- Тестовая программа (программа №2), которая загружает библиотеки, используя только их местоположение и контракты.

Задание 6. Необходимо вычислить значение числа e , являющегося основанием натурального логарифма, с использованием двух методов: предельного перехода $(1+1/x)^x$ и суммы ряда $\sum_{n=0}^{\infty} 1/(n!)$. Реализация должна быть выполнена в виде функции `Float E(int x)`.

Задание 8. Реализовать функцию `Char* translation(long x)` для перевода числа xxx из десятичной системы счисления в другую. В частности, необходимо обеспечить перевод в двоичную и тричную системы счисления.

Общие сведения о программе

В ходе выполнения лабораторной работы были изучены и реализованы два подхода к работе с библиотеками: статическая линковка и динамическая загрузка. В первой программе использовалась статическая линковка, что позволяло напрямую вызывать функции из библиотеки `lib1` без необходимости их загрузки во время выполнения. Вторая программа реализовывала динамическую загрузку библиотек, обеспечивая возможность

переключения между lib1 и lib2 во время работы программы. Реализованный функционал включал вычисление приближенного значения e двумя способами, где в lib1 использовалась формула $(1 + 1/x)^x$, а в lib2 расчет выполнялся с использованием суммы ряда $1/p!$. Также была добавлена возможность перевода чисел в различные системы счисления, где lib1 поддерживала перевод в двоичную систему, а lib2 – в троичную. Программа корректно обрабатывала пользовательский ввод и исключительные ситуации, что обеспечивало надежность ее выполнения. Использование динамической загрузки библиотек позволило расширить функциональные возможности приложения без необходимости изменения исходного кода. Лабораторная работа помогла освоить принципы работы со статическими и динамическими библиотеками, а также механизмы эффективной обработки данных и взаимодействия между процессами.

Код программы

program1.cpp

```
// статическая линковка
#include <iostream>
#include "functions.hpp"

int main() {
    std::cout << "Программа №1 (статическая линковка библиотеки: lib1)\n";

    while (true) {
        std::cout << "\nВведите команду:\n";
        std::cout << "1 x - вычислить e\n";
        std::cout << "2 x - перевести число в двоичную систему\n";
        std::cout << "3 - выйти\n";

        int command;
        std::cin >> command;

        if (command == 3) {
            // выход
            std::cout << "Завершение работы.\n";
            break;
        }

        if (command == 1) {
            int x;
            std::cin >> x;
            float result = E_calc(x);
            if (result < 0) {
                std::cout << "Ошибка вычисления. Возможно, x <= 0.\n";
            }
        }
    }
}
```

```

    } else {
        std::cout << "e(" << x << ") = " << result << "\n";
    }
} else if (command == 2) {
    long x;
    std::cin >> x;
    char* result = translation(x);
    std::cout << "Число в двоичной системе: " << result << "\n";
} else {
    std::cout << "Неверная команда\n";
}
}

return 0;
}

```

program2.cpp

```

// динамическая загрузка
#include <iostream>
#include <dlfcn.h>
#include <cstdlib> // для getenv
#include <cstring> // для ошибок dLError()

// типы указателей на функции
using EFunc = float(*)(int);
using TranslationFunc = char(*)(long);

int main() {
    std::cout << "Программа №2 (динамическая загрузка библиотек)\n";

    const char* pathToLib1 = std::getenv("PATH_TO_LIB1");
    if (!pathToLib1) {
        std::cerr << "Переменная окружения PATH_TO_LIB1 не установлена\n";
        return 1;
    }
    const char* pathToLib2 = std::getenv("PATH_TO_LIB2");
    if (!pathToLib2) {
        std::cerr << "Переменная окружения PATH_TO_LIB2 не установлена\n";
        return 1;
    }

    // сохраняю пути в массив
    const char* lib_paths[] = { pathToLib1, pathToLib2 };
    int current_lib = 0; // по умолчанию грузю lib1

    // грузю библиотеку
    void* handle = dlopen(lib_paths[current_lib], RTLD_LAZY);
    //dlopen(const char* filename, int flag) LAZY загружать символы по мере
    необходимости.
}

```

```

if (!handle) {
    std::cerr << "Ошибка загрузки библиотеки: " << dlerror() << "\n";
    return 1;
}

// ищем символы
//      reinterpret_cast- приведение типа
//      dlsym - ищет символы в библиотеках динамических
EFunc E_calc = reinterpret_cast<EFunc>(dlsym(handle, "E_calc"));
TranslationFunc translation = reinterpret_cast<TranslationFunc>(dlsym(handle,
"translation"));

char* error = dlerror();
if (error) {
    std::cerr << "Ошибка получения символа: " << error << "\n";
    dlclose(handle);
    return 1;
}

// основной цикл ввода команд
while (true) {
    std::cout << "\nТекущая библиотека: " << lib_paths[current_lib] << "\n";
    std::cout << "Введите команду:\n";
    std::cout << "0 - переключить реализацию (между lib1 и lib2)\n";
    std::cout << "1 x - вычислить e^n\n";
    std::cout << "2 x - перевести число в другую систему счисления\n";
    std::cout << "3 - выйти\n";

    int command;
    std::cin >> command;

    if (command == 3) {
        std::cout << "Завершение работы.\n";
        break;
    }

    if (command == 0) {
        // переключаемся между библиотеками
        dlclose(handle); // закрываем предыдущую
        current_lib = 1 - current_lib; // переключаемся
        handle = dlopen(lib_paths[current_lib], RTLD_LAZY);
        if (!handle) {
            std::cerr << "Ошибка загрузки библиотеки: " << dlerror() << "\n";
            return 1;
        }
        E_calc = reinterpret_cast<EFunc>(dlsym(handle, "E_calc"));
        translation = reinterpret_cast<TranslationFunc>(dlsym(handle, "translation"));
        error = dlerror();
        if (error) {
            std::cerr << "Ошибка получения символа: " << error << "\n";
            dlclose(handle);
            return 1;
        }
    }
}

```

```

    }
}
else if (command == 1) {
    // вычислить e
    int x;
    std::cin >> x;
    float result = E_calc(x);
    if (result < 0) {
        std::cout << "Ошибка вычисления e. Возможно x <= 0 для первой реализации "
            "или x < 0 для второй.\n";
    } else {
        std::cout << "e(" << x << ") = " << result << "\n";
    }
}
else if (command == 2) {
    // перевести в другую систему счисления
    long x;
    std::cin >> x;
    char* result = translation(x);
    std::cout << "Перевод числа " << x << " = " << result << "\n";
}
else {
    std::cout << "Неверная команда\n";
}
}

dlclose(handle);
return 0;
}

```

src/lib1.cpp

```

#include <cmath>
#include <algorithm>
#include "functions.hpp"

//e(x) = (1 + 1/x)^x
float E_calc(int x) {
    if (x <= 0) {
        return -1.0f;
    }
    // static_cast оператор явного присвоения типов
    double xd = static_cast<double>(x);
    return std::pow(1.0 + 1.0/xd, xd);
}
//перевод в двоичную систему счисления
char* translation(long x) {
    static char buffer[65]; // результа в виде массива
    int i = 0;

```

```

//случай когда x=0
if (x == 0) {
    buffer[i++] = '0';
    buffer[i] = '\0';
    return buffer;
}
if (x < 0) {
    // Простая заглушка: вернём "-1"
    // Или можно добавить логику перевода отрицательных чисел
    static char errBuf[] = "-1";
    return errBuf;
}
while (x > 0) {
    buffer[i++] = (x % 2) + '0';
    x /= 2;
}
buffer[i] = '\0';
// перевернул строку
for (int j = 0, k = i - 1; j < k; ++j, --k) {
    std::swap(buffer[j], buffer[k]);
}
return buffer;
}

```

src/lib2.cpp

```

#include <cmath>
#include <algorithm>
#include "functions.hpp"

static double factorial(int n) {
    // факториал
    if (n < 0) return 0.0;
    double f = 1.0;
    for (int i = 1; i <= n; ++i) {
        f *= i;
    }
    return f;
}

////e(x) = sum_{p=0..x} (1/p!)
float E_calc(int x) {
    if (x < 0) {
        return -1.0f;
    }
    // считаю сумму 1/p! от p = 0 до p = x
    double sum = 0.0;
    for (int p = 0; p <= x; p++) {
        double fact = factorial(p);
        if (fact == 0.0) {
            // на всякий случай от ошибок

```

```

        return -1.0f;
    }
    sum += 1.0 / fact;
}
return static_cast<float>(sum); //привёл к float
}

//перевод в троичную систему счисления
char* translation(long x) {
    static char buffer[65];
    int i = 0;

    if (x == 0) {
        buffer[i++] = '0';
        buffer[i] = '\0';
        return buffer;
    }

    if (x < 0) {
        static char errBuf[] = "-1";
        return errBuf;
    }

    while (x > 0) {
        buffer[i++] = (x % 3) + '0';
        x /= 3;
    }
    buffer[i] = '\0';

    for (int j = 0, k = i - 1; j < k; ++j, --k) {
        std::swap(buffer[j], buffer[k]);
    }
    return buffer;
}

```

Пример вывода:

```

user@DESKTOP-
Q8E0OMG:/mnt/c/Users/user/Desktop/Labs/os/lr4/build$ ./test_lib1
[=====] Running 5 tests from 2 test suites.
[-----] Global test environment set-up.
[-----] 2 tests from Lib1_Ecalc
[ RUN      ] Lib1_Ecalc.PositiveX
[          OK ] Lib1_Ecalc.PositiveX (0 ms)
[ RUN      ] Lib1_Ecalc.ZeroOrNegativeX
[          OK ] Lib1_Ecalc.ZeroOrNegativeX (0 ms)
[-----] 2 tests from Lib1_Ecalc (0 ms total)

[-----] 3 tests from Lib1_Translation
[ RUN      ] Lib1_Translation.Zero

```



```

[      OK ] Lib1_Translation.Zero (0 ms)
[ RUN      ] Lib1_Translation.PositiveNumber
[      OK ] Lib1_Translation.PositiveNumber (0 ms)
[ RUN      ] Lib1_Translation.NegativeNumber
[      OK ] Lib1_Translation.NegativeNumber (0 ms)
[-----] 3 tests from Lib1_Translation (0 ms total)

[-----] Global test environment tear-down
[=====] 5 tests from 2 test suites ran. (0 ms total)
[ PASSED ] 5 tests.

```

Выводы

В ходе выполнения лабораторной работы была реализована программа, демонстрирующая принципы работы с динамическими библиотеками и их использование двумя способами: через статическую линковку и динамическую загрузку во время исполнения. В рамках задания была разработана функциональность для вычисления приближенного значения числа e двумя различными методами, а также перевода числа из десятичной системы счисления в другую. Первая реализация вычисления e использует формулу $(1 + 1/x)^x$, тогда как вторая основана на сумме ряда $1/n!$. Перевод числа осуществляется в двоичную систему в первой библиотеке и в троичную – во второй.

Программа была представлена в двух вариантах: первая версия компилируется с одной из библиотек на этапе линковки, в то время как вторая позволяет загружать библиотеки динамически и переключаться между ними без перезапуска. Это дало возможность провести анализ двух подходов к использованию библиотек: статическая линковка обеспечивает большую производительность, но жестко привязывает программу к конкретной реализации, тогда как динамическая загрузка предоставляет гибкость и возможность менять алгоритмы выполнения на лету.